

UVSim milestone 2 design

Purpose and scope

UVSim is a command-line BasicML simulator for students learning machine language and computer architecture. A student selects a program file, supplies values requested by READ instructions, and observes values printed by WRITE instructions. The simulator executes all 12 required operations until HALT or an error.

The prototype uses Python 3.10 or newer and the standard library. It has no GUI or external package dependencies. See README.txt for installation, launch commands, and step-by-step prototype checks.

User stories

1. As a computer science student, I want to load and execute a BasicML program so I can observe how instructions manipulate memory and the accumulator.
2. As a computer science student, I want to enter values through the console, see program output, and receive errors for invalid programs or input so I can test and correct my work.

Architecture

Component	Responsibility
main.py	Accept a filename argument or prompt for one, create the machine, load and run the program, and report errors.
loader.py	Read the program file and return validated integer words in order without modifying machine state.
uvsim.py	Own machine state, initialize memory, fetch and decode instructions, dispatch handlers, and control execution.
operations/io_memory.py	Implement READ, WRITE, LOAD, and STORE.
operations/arithmetic.py	Implement ADD, SUBTRACT, DIVIDE, and MULTIPLY, including sign-preserving overflow truncation.
operations/control_flow.py	Implement BRANCH, BRANCHNEG, BRANCHZERO, and HALT.

Separating file parsing, execution, and operation handlers allows each area to be maintained independently. Future operations can be added through handlers and the dispatch mapping without moving file or console orchestration into arithmetic code.

Machine state

State	Meaning and initial value
Memory	100 integer words, addressed 00–99, initially zero. Each location may hold an instruction, data, or unused space.
Accumulator	Separate integer register for calculations, initially zero.
Instruction counter	Address of the next instruction, initially 00.
Instruction register	Most recently fetched instruction, initially zero.
Halted flag	Whether HALT has stopped execution, initially false.

Words range from -9999 through 9999. File instructions use a plus sign; data may use either sign. An instruction's first two digits are its opcode and its last two digits are its operand address. For example, +2007 loads memory address 07 into the accumulator. Instructions and data share memory; STORE and READ can replace words that were originally instructions.

Loading and execution

1. The CLI selects a filename and calls `load_program(path)`.
2. The loader validates word formatting and capacity, then returns integers.
It does not validate opcodes because files also contain data words.
3. `UVSim.load(words)` checks capacity and numeric range before resetting the machine. It copies the words starting at 00 and zeroes unused memory.
4. `run()` repeatedly calls `step()` until the halted flag is true.
5. `step()` checks the counter, fetches the instruction, rejects negative instructions, and decodes opcode and operand with `divmod(word, 100)`.
An unsupported opcode raises an error.
6. Before dispatch, `step()` advances the counter by one. A taken branch replaces that value with its destination. An untaken branch leaves it unchanged, so execution continues sequentially. A branch may target its own address.
7. Handlers receive the machine and operand. `READ` also receives `sys.stdin`; `step()` prints the keyboard prompt before calling it. Handler imports occur inside `step()` to avoid circular imports with modules that import `UVSim`.
8. `HALT` sets the halted flag. Later calls to `step()` do nothing while halted.
A successful CLI run exits with status 0.

Use cases

The primary actor for every use case is the student running a BasicML program. For UC-03 through UC-13, the shared precondition is that a program has been loaded, the machine is not halted, and execution has reached the relevant instruction with an operand address in 00–99. Unless specified otherwise, a successful instruction continues to the next address. A reported runtime error ends the CLI run with status 1 rather than requesting another value.

The identifiers below are retained for references from tests and the unit-test spreadsheet.

UC-01 — Select and load a BasicML file

- **Precondition:** The student has launched the CLI and has a program filename.
- **Trigger:** The student supplies a filename argument or enters one at the prompt.

Main flow:

1. Read and validate the file through UC-02, then load its words starting at address 00.
2. Reset the accumulator, instruction counter/register, and halted flag before execution begins.

- **Alternate/error flow:** An empty prompted filename, unavailable file, or invalid file contents produces an error. No partial program is loaded.

- **Postcondition:** A valid program is in memory, unused words are zero, and the machine is ready to execute from 00; otherwise the CLI reports an error.

UC-02 — Validate words and program capacity

- **Precondition:** A program file can be opened for reading.

- **Trigger:** The loader processes the file's lines.

Main flow:

1. Strip surrounding whitespace, skip blank lines, require a sign and exactly four decimal digits per remaining line, and convert to integers.

2. Accept at most 100 words in the range -9999 through 9999.

- **Alternate/error flow:** Malformed words or excess capacity raise `ValueError`.

An empty or all-blank file returns an empty list; executing the resulting zero-filled memory subsequently fails with opcode 00.

- **Postcondition:** Return the complete validated list or an error without modifying machine memory.

UC-03 — READ console input into memory (10)

- **Trigger:** Execution reaches `READ`.

Main flow:

1. Display a prompt, read one console line, convert it to an integer, check the word range, and store the value at the operand address.

- **Alternate/error flow:** Nonnumeric or out-of-range input raises `ValueError`.

Blank input or end of input raises `EOFError`. These failures do not change the destination. Keyboard input may be 7, -5, or +0007; it does not require the fixed-width file format.

- **Postcondition:** The destination contains the accepted input; the accumulator is unchanged.

UC-04 — WRITE memory to console (11)

- **Trigger:** Execution reaches `WRITE`.

Main flow:

1. Print the operand memory word with a newline using four zero-padded magnitude digits and a separate minus sign for negative values.

- **Alternate flow:** Negative values retain their minus sign. For example, 12 prints as 0012, -5 as -0005, and zero as 0000.

- **Postcondition:** One value is printed; memory and accumulator are unchanged.

UC-05 — LOAD accumulator (20)

- **Trigger:** Execution reaches `LOAD`.

Main flow:

1. Copy the operand memory word into the accumulator.

- **Alternate flow:** Zero and negative values are copied without changing their sign.

- **Postcondition:** The accumulator holds the selected word; memory is unchanged.

UC-06 — STORE accumulator (21)

- **Trigger:** Execution reaches STORE.

Main flow:

1. Copy the accumulator into the operand memory address, replacing its previous word.

- **Alternate flow:** The destination may previously have held data, an instruction, or an unused zero word.

- **Postcondition:** The destination equals the accumulator; the accumulator is unchanged.

UC-07 — ADD (30)

- **Trigger:** Execution reaches ADD.

Main flow:

1. Add the operand memory word to the accumulator and retain the result in the accumulator.

- **Alternate/error flow:** Zero and negative operands are valid. A result outside -9999 through 9999 discards higher-order magnitude digits, retains the sign, and continues.

- **Postcondition:** The accumulator contains the valid sum; memory is unchanged.

UC-08 — SUBTRACT (31)

- **Trigger:** Execution reaches SUBTRACT.

Main flow:

1. Subtract the operand memory word from the accumulator and retain the result in the accumulator.

- **Alternate/error flow:** Negative operands and negative results are valid.

A result outside the word range is truncated to four magnitude digits, retaining the sign.

- **Postcondition:** The accumulator contains the valid difference; memory is unchanged.

UC-09 — DIVIDE (32)

- **Trigger:** Execution reaches DIVIDE.

Main flow:

1. Divide the accumulator by the operand memory word.

2. Truncate any fractional part toward zero and retain the integer quotient in the accumulator.

- **Alternate/error flow:** For example, -7 divided by 3 yields -2. A zero divisor

raises `ValueError` and preserves the accumulator. The integer result is stored using the same sign-preserving four-digit truncation policy.

- **Postcondition:** The accumulator contains the valid quotient; memory is unchanged.

UC-10 — MULTIPLY (33)

- **Trigger:** Execution reaches MULTIPLY.

Main flow:

1. Multiply the accumulator by the operand memory word and retain the result in the accumulator.

- **Alternate/error flow:** A zero operand produces zero. Negative operands are valid. A result outside the word range discards higher-order magnitude digits and retains the sign; execution continues.

- **Postcondition:** The accumulator contains the valid product; memory is unchanged.

UC-11 — BRANCH (40)

- **Trigger:** Execution reaches BRANCH.

Main flow:

1. Replace the instruction counter with the operand address, regardless of the accumulator's value.

- **Alternate flow:** The target can be earlier, later, or the current instruction.

Repeated branches can create a loop.

- **Postcondition:** The next instruction is fetched at the target address; memory and accumulator are unchanged.

UC-12 — BRANCHNEG (41)

- **Trigger:** Execution reaches BRANCHNEG.

Main flow:

1. If the accumulator is negative, replace the instruction counter with the operand address.

- **Alternate flow:** If the accumulator is zero or positive, keep the sequential next address already set by step ().

- **Postcondition:** Execution continues at the selected address; memory and accumulator are unchanged.

UC-13 — BRANCHZERO (42)

- **Trigger:** Execution reaches BRANCHZERO.

Main flow:

1. If the accumulator equals zero, replace the instruction counter with the operand address.

- **Alternate flow:** If the accumulator is positive or negative, keep the sequential next address already set by step ().

- **Postcondition:** Execution continues at the selected address; memory and accumulator are unchanged.

UC-14 — Execute a program and HALT (43)

- **Precondition:** A program has been loaded and the machine is ready to execute.

- **Trigger:** The CLI calls run ().

Main flow:

1. Repeatedly fetch, decode, and dispatch instructions.

2. On opcode 43, set the halted flag and return from the loop.

3. HALT ignores the operand; the sample programs use +4300.

- **Alternate/error flow:** A negative instruction, unsupported opcode, counter outside 00–99, or handler error stops execution. A program without HALT may encounter an error or loop indefinitely; there is no automatic instruction limit. The student can interrupt with Ctrl+C.

- **Postcondition:** On HALT, execution stops and final machine state is retained.

The CLI exits with status 0. On a handled error, it reports the error and exits with status 1.

Error handling and implementation policies

The assignment defines the machine size, word format, loading address, and opcode behavior. The following details describe the prototype's implementation choices:

- File lines permit surrounding whitespace and blank lines. Console READ uses

Python integer conversion followed by word-range validation.

- Fractional division truncates toward zero. Overflow discards higher-order magnitude digits while preserving the sign (12345 becomes 2345; -12345 becomes -2345). Division by zero remains an error.

- UVSim. Load validates capacity and numeric range before resetting state.

A rejected load preserves the previous state.

- A fetched instruction is recorded in the instruction register even if it is subsequently rejected. When a handler raises `ValueError` or `EOFError`, `step()` restores the counter to the failing address. It does not perform a general rollback of earlier instructions or console output.

- Arithmetic and branch handlers validate operand bounds. I/O/memory handlers rely on the 00–99 operand obtained from a valid four-digit instruction.

- `main.py` reports handled file, validation, and end-of-input errors to standard error. Ctrl+C is a user interruption and may display a Python traceback.

- Loading does not require HALT or analyze control flow. Execution determines whether the program reaches HALT, encounters an error, or loops.

Verification and traceability

The unit-test spreadsheet references the UC-01 through UC-14 identifiers above. Automated tests are stored in `tests/` and run with `python -m unittest discover -s tests -v`.

`Test1.txt` exercises input, addition, storage, output, and HALT. `Test2.txt` uses subtraction and `BRANCHNEG` to print the larger of two inputs. The additional `Multiply`, `Divide`, `Branch`, and `BranchZero` sample programs exercise the remaining opcodes; commands and expected outputs are documented in `programs/README.txt`. Manual sample checks complement the unit tests and are not additional unit-test methods.

Deferred Milestone 3 work

GUI implementation and recoverable GUI READ/file errors remain future tasks. The current console application still reports errors and exits; these feedback items are not marked complete. The revised PDF uses the separate UC-01 through UC-14 identifiers already used here and in the test register.